

Java Backend Architecture

Interview Series

Chapter 8.1

TLS/HTTPS & X.509

Certificates

50+ Interview Q&A

Code Examples

Architecture Diagrams

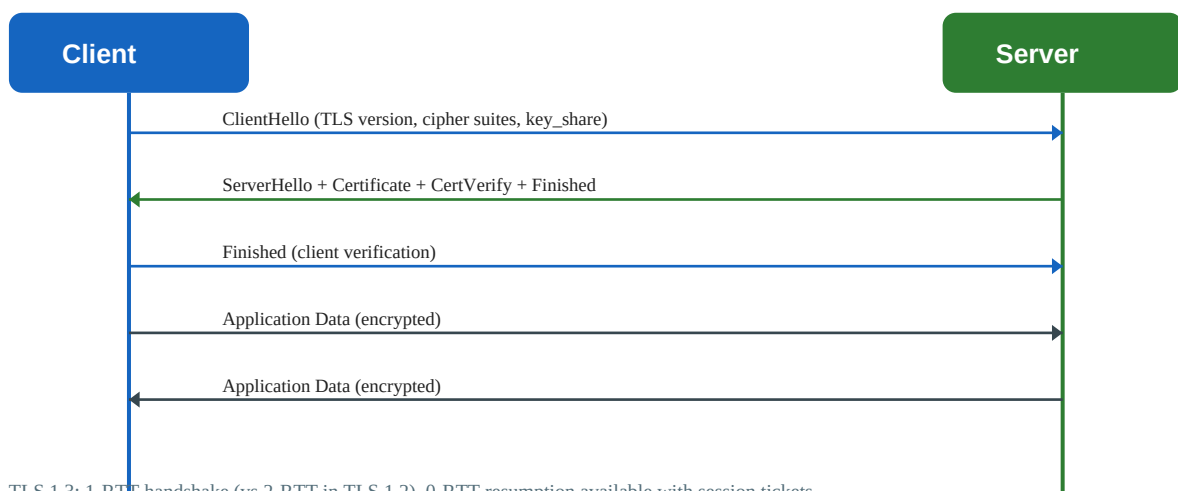
Advanced Topics

Chapter 8.1 — TLS/HTTPS & X.509 Certificates

Introduction

Transport Layer Security (TLS) is the cryptographic protocol that secures HTTPS connections. TLS 1.3 (RFC 8446) is the current standard, offering a faster 1-RTT handshake, forward secrecy by default, and removal of weak cipher suites. X.509 certificates bind public keys to identities through a chain of trust. Understanding TLS internals — handshake mechanics, certificate validation, and mTLS — is critical for backend security architecture.

TLS 1.3 Handshake



TLS 1.3 reduces the handshake to **1 Round Trip Time (1-RTT)**: the client sends its key share in ClientHello; the server can respond with encrypted data immediately. 0-RTT resumption allows the client to send data in the first flight using a pre-shared key from a previous session, but is vulnerable to replay attacks and should be used carefully.

Step	Who Sends	Content	Purpose
ClientHello	Client	TLS version, cipher suites, key_share (ECDHE, AES, etc.)	Offer capabilities, initiate key exchange
ServerHello	Server	Chosen cipher, key_share	Finalize key exchange
Certificate	Server	X.509 cert chain	Prove server identity
CertificateVerify	Server	Signature over handshake transcript	Prove private key ownership
Finished	Server	MAC of handshake	Authenticate handshake
Finished	Client	MAC of handshake	Confirm handshake integrity

Symmetric vs Asymmetric Encryption in TLS

Type	Algorithm	Use in TLS	Key Size
Asymmetric	RSA-2048/4096	Certificate signatures	2048+ bits
Asymmetric	ECDSA P-256/P-384	Certificate signatures (modern)	256/384 bits

Key Exchange	ECDH (X25519)	Session key derivation	256 bits
Symmetric	AES-128/256-GCM	Bulk data encryption (AEAD)	128/256 bits
Symmetric	ChaCha20-Poly1305	Mobile/low-power encryption	256 bits
Hash	SHA-256/384	PRF, HMAC, cert signing	256/384 bits

X.509 Certificate Fields



Browser verifies: leaf → intermediate → root (which is in trust store). OCSP/CRL checks for revocation.

Field	Description	Example
Subject CN	Common Name — primary hostname	example.com
Subject Alt Name	SANs — all valid hostnames (replaces CN)	example.com, *.example.com
Issuer	CA that signed this certificate	DigiCert Global Root
Validity	notBefore / notAfter dates	2024-01-01 to 2025-01-01
Public Key	Certificate's public key algorithm + value	RSA 2048 or ECDSA P-256
Key Usage	Permitted operations	digitalSignature, keyEncipherment
Extended Key Usage	Purpose constraints	serverAuth, clientAuth
Serial Number	Unique identifier from CA	7a:3b:cc:...
Signature Algorithm	Algorithm used to sign certificate	sha256WithRSAEncryption
OCSP URL	Online Certificate Status Protocol URL	http://ocsp.digicert.com

Certificate Pinning

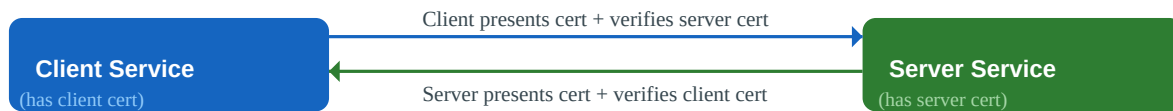
Certificate pinning associates a specific certificate or public key with a service, preventing MITM attacks even with a rogue CA. **HPKP** (HTTP Public Key Pinning) is deprecated — brittle and causes outages. Modern approach: use **certificate transparency (CT) logs** monitoring and pin at the application level (e.g., in OkHttp or Java TrustManager) for mobile apps.

```
// Java: Custom TrustManager with certificate pinning
public class PinnedTrustManager extends
X509ExtendedTrustManager {
    private static final String EXPECTED_PIN = // SHA-256 of
SubjectPublicKeyInfo "sha256/AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=";
    @Override public
void checkServerTrusted(X509Certificate[] chain, String authType,
SSLSocket socket) throws
CertificateException {
        // 1. Standard chain validation
        defaultTrustManager.checkServerTrusted(chain, authType, socket);
        // 2. Pin check
        String pin = computePin(chain[0].getPublicKey());
        if (!EXPECTED_PIN.equals(pin)) {
            throw new CertificateException("Certificate pin mismatch!");
        }
    }
    private String computePin(PublicKey key) throws CertificateException {
        try {
            MessageDigest md =

```

```
MessageDigest.getInstance("SHA-256")); byte[] hash = md.digest(key.getEncoded()); return
"sha256/" + Base64.getEncoder().encodeToString(hash); } catch (NoSuchAlgorithmException e) {
throw new CertificateException(e); } } }
```

mTLS (Mutual TLS)



mTLS: both parties authenticate each other — used for service-to-service in zero-trust architectures

```
# Spring Boot mTLS configuration (application.yaml)
server:
  ssl:
    enabled: true
    key-store: classpath:server-keystore.p12
    key-store-password: ${SSL_KEYSTORE_PASSWORD}
    key-store-type: PKCS12
    trust-store: classpath:truststore.p12
    trust-store-password: ${SSL_TRUSTSTORE_PASSWORD}
    trust-store-type: PKCS12
    client-auth: need # NEED = require client cert; WANT = optional
# Generate self-signed certs for mTLS (development)
# Server cert
keytool -genkeypair -alias server -keyalg RSA -keysize 2048 \
  -validity 365 -keystore server-keystore.p12 -storetype PKCS12 \
  -dn "CN=localhost, OU=Dev, O=Example, C=US"
# Export server cert to trust store
keytool -exportcert -alias server -keystore server-keystore.p12 \
  -file server.crt
keytool -importcert -alias server -file server.crt \
  -keystore client-truststore.p12 -storetype PKCS12 -noprompt
```

OCSP Stapling

OCSP (Online Certificate Status Protocol) allows clients to check if a certificate is revoked. Without stapling, the client must contact the CA's OCSP server — adding latency and leaking browsing behaviour.

OCSP stapling has the server include a pre-fetched OCSP response in the TLS handshake, eliminating the client's need to contact the CA. Enable in Nginx: `ssl_stapling on;` `ssl_stapling_verify on;`

TLS Configuration in Spring Boot (Nginx)

```
# Nginx - TLS 1.3 with modern cipher suites
server {
  listen 443 ssl;
  server_name example.com;
  ssl_certificate /etc/letsencrypt/live/example.com/fullchain.pem;
  ssl_certificate_key /etc/letsencrypt/live/example.com/privkey.pem;
  ssl_protocols TLSv1.2 TLSv1.3;
  ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:
  TLS_AES_128_GCM_SHA256:TLS_AES_256_GCM_SHA384;
  ssl_prefer_server_ciphers off;
  # Let client pick (TLS 1.3 ignores anyway)
  # HSTS: force HTTPS for 1 year, include subdomains
  add_header Strict-Transport-Security "max-age=31536000; includeSubDomains; preload";
  # OCSP stapling
  ssl_stapling on;
  ssl_stapling_verify on;
  ssl_trusted_certificate /etc/letsencrypt/live/example.com/chain.pem;
  resolver 8.8.8.8 valid=300s;
}
```

50+ Interview Questions & Answers

Q1. What is TLS and why is it important?

Transport Layer Security encrypts data in transit, authenticates the server (and optionally client), and ensures data integrity. It prevents eavesdropping, tampering, and man-in-the-middle attacks on network communication.

Q2. What is the difference between SSL and TLS?

SSL (Secure Sockets Layer) is the deprecated predecessor to TLS. SSL 3.0, TLS 1.0, and TLS 1.1 are all deprecated. TLS 1.2 is still widely used; TLS 1.3 is the current standard (RFC 8446, 2018).

Q3. How does TLS 1.3 differ from TLS 1.2?

TLS 1.3: 1-RTT handshake (vs 2-RTT), mandatory forward secrecy (only ECDHE key exchange), removed weak ciphers (RSA key exchange, DH, 3DES, RC4), 0-RTT resumption, encrypted handshake (certificates encrypted).

Q4. What is forward secrecy?

A property ensuring that compromise of the server's long-term private key does not compromise past session keys. Achieved by using ephemeral key exchange (ECDHE) — each session generates new keys that are discarded after use.

Q5. What is symmetric vs asymmetric encryption in TLS?

Asymmetric (RSA/ECDSA): used to authenticate and exchange keys during handshake — slow, computationally expensive. Symmetric (AES-GCM, ChaCha20): used to encrypt actual data — fast. TLS uses asymmetric to establish symmetric session keys.

Q6. What is ECDH key exchange?

Elliptic Curve Diffie-Hellman — both parties exchange public keys derived from elliptic curve math. Neither party sends their private key. They independently compute the same shared secret. In TLS 1.3, ephemeral ECDHE is the only key exchange method.

Q7. What is an X.509 certificate?

A digital certificate binding a public key to an identity (domain name, organisation). Contains: subject, issuer, validity period, public key, extensions (SAN, key usage), and the CA's digital signature.

Q8. What is the difference between CN and SAN in a certificate?

CN (Common Name) was the original field for the hostname. SAN (Subject Alternative Name) is the modern standard — allows multiple hostnames and IPs. Browsers now require SAN and ignore CN for host verification.

Q9. What is a certificate chain (trust chain)?

Root CA → Intermediate CA → Leaf certificate. Each certificate is signed by the one above. The root CA is self-signed and pre-installed in OS trust stores. Browsers verify the full chain from leaf to trusted root.

Q10. Why use intermediate CAs?

Root CAs are kept offline for security. Intermediate CAs handle day-to-day certificate issuance. If an intermediate CA is compromised, it can be revoked without invalidating the root.

Q11. What is OCSP?

Online Certificate Status Protocol — allows clients to check if a certificate has been revoked. Client contacts the CA's OCSP endpoint with the certificate's serial number and receives good/revoked/unknown response.

Q12. What is OCSP stapling?

The server pre-fetches and caches the OCSP response from the CA, then includes (staples) it in the TLS handshake. Eliminates client latency of contacting CA, improves privacy (CA doesn't learn about client connections).

Q13. What is certificate revocation and why is it needed?

Marks a certificate as invalid before its expiry — used when the private key is compromised, the domain changes ownership, or the certificate was mis-issued. Mechanisms: CRL (certificate revocation list) and OCSP.

Q14. What is certificate pinning?

Associating a service with a specific certificate or public key. If the server presents a different certificate (even a valid CA-issued one), the connection is rejected. Prevents MITM with rogue CA certificates.

Q15. Why is HPKP (HTTP Public Key Pinning) deprecated?

HPKP sent pinned keys in HTTP headers, but a misconfiguration could permanently lock users out of a site. One bad deployment could break HTTPS permanently for all users who cached the header. HPKP was removed from Chrome in 2018.

Q16. What is Certificate Transparency (CT)?

A public, append-only log of all issued certificates. CAs must submit certificates to CT logs. Browsers verify certificates have valid CT proof (SCTs). Allows domain owners to detect mis-issued certificates quickly.

Q17. What is SNI (Server Name Indication)?

A TLS extension where the client includes the hostname in ClientHello. Allows a single IP address to serve multiple HTTPS domains — the server uses SNI to select the correct certificate before the handshake completes.

Q18. What is mTLS (Mutual TLS)?

Both client and server authenticate each other using certificates. Server presents its certificate (as in standard TLS), and client also presents a certificate that the server validates. Used for service-to-service authentication.

Q19. When is mTLS used?

Service mesh authentication (Istio, Linkerd), API client authentication, microservices in zero-trust architectures, payment systems, and IoT device authentication — anywhere mutual identity verification is required.

Q20. What is the difference between client-auth: need vs want?

need: server requires client certificate — rejects connections without valid client cert. want: server requests client cert but proceeds if client doesn't provide one (optional client authentication).

Q21. What is a PKCS12 keystore?

A binary format (file extension .p12 or .pfx) that stores a private key + certificate chain in a single password-protected file. Commonly used in Java (keytool) for storing TLS credentials.

Q22. What is JKS vs PKCS12?

JKS (Java KeyStore) is Java's proprietary format. PKCS12 is the standard format (RFC 7292). Java 9+ defaults to PKCS12. PKCS12 is preferred — portable across platforms and languages.

Q23. How does Let's Encrypt automate certificate issuance?

Let's Encrypt uses the ACME protocol. The client (Certbot) proves domain control via HTTP-01 (place a file at /.well-known/acme-challenge/) or DNS-01 (create a DNS TXT record). Issues 90-day certificates, auto-renewed by the ACME client.

Q24. What is HSTS (HTTP Strict Transport Security)?

HTTP response header instructing browsers to only connect via HTTPS for a specified duration: Strict-Transport-Security: max-age=31536000; includeSubDomains; preload. Prevents SSL stripping attacks.

Q25. What is SSL stripping?

A MITM attack where the attacker downgrades an HTTPS connection to HTTP by intercepting the initial HTTP request before it's redirected to HTTPS. HSTS and HTTPS-only cookies prevent this attack.

Q26. What is the TLS session ticket?

A server-encrypted blob containing session state, sent to the client. The client presents it in future connections to resume a TLS session without a full handshake (TLS 1.2). TLS 1.3 uses PSK (pre-shared key) resumption instead.

Q27. What are AEAD ciphers in TLS?

Authenticated Encryption with Associated Data — provides both confidentiality (encryption) and integrity (authentication) in a single operation. AES-GCM and ChaCha20-Poly1305 are AEAD ciphers. All TLS 1.3 ciphers are AEAD.

Q28. What is a wildcard certificate?

A certificate with SAN *.example.com — valid for all direct subdomains (mail.example.com, api.example.com) but NOT sub-subdomains (a.b.example.com). Cannot be used for the root domain (example.com) unless listed separately.

Q29. How do you verify the TLS configuration of a server?

openssl s_client -connect example.com:443 -tls1_3. Tools: testssl.sh, SSL Labs (ssllabs.com/ssltest), nmap --script ssl-enum-ciphers. Check for: TLS version, cipher suites, certificate validity, HSTS, OCSP stapling.

Q30. What is a PEM file?

Base64-encoded DER certificate/key wrapped in -----BEGIN CERTIFICATE----- markers. The standard format for storing certificates and private keys in files. Multiple PEM blocks can be in one file (certificate chain in fullchain.pem).

Q31. What is key usage extension?

Certificate field limiting what the key can be used for: digitalSignature (signing), keyEncipherment (RSA key exchange), keyAgreement (DH/ECDH). Extended Key Usage adds purpose: serverAuth, clientAuth, codeSigning, emailProtection.

Q32. What cipher suites does TLS 1.3 support?

Only 5 cipher suites, all AEAD: TLS_AES_128_GCM_SHA256, TLS_AES_256_GCM_SHA384, TLS_CHACHA20_POLY1305_SHA256, TLS_AES_128_CCM_SHA256, TLS_AES_128_CCM_8_SHA256. No RSA key exchange (all use ECDHE).

Q33. What is the difference between DV, OV, and EV certificates?

DV (Domain Validated): CA only verifies domain control — automated, minutes. OV (Organization Validated): CA verifies organisation identity — days. EV (Extended Validation): rigorous verification, shows org name in old browsers — weeks.

Q34. What is certificate transparency SCT?

Signed Certificate Timestamp — proof that a certificate was submitted to a CT log. TLS 1.3 requires SCTs via the certificate extension, OCSP stapling, or TLS extension. Chrome requires SCTs for publicly trusted certificates.

Q35. How does Java validate TLS certificates?

Java uses the JSSE (Java Secure Socket Extension) with its own trust store (cacerts in \$JAVA_HOME/lib/security). The default X509TrustManager validates the cert chain against cacerts. Override by providing a custom TrustManager or KeyStore.

Q36. What is SSLContext in Java?

Factory for SSLSocket and SSLEngine. Configure it with KeyManager (server's private key + cert) and TrustManager (trusted CAs). Example: `SSLContext ctx = SSLContext.getInstance('TLS');`
`ctx.init(keyManagers, trustManagers, new SecureRandom());`